



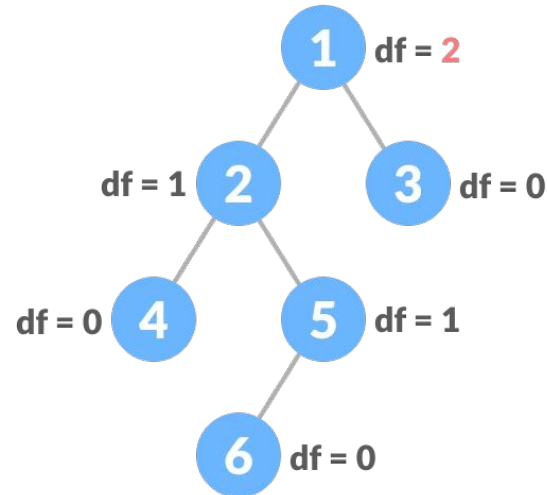
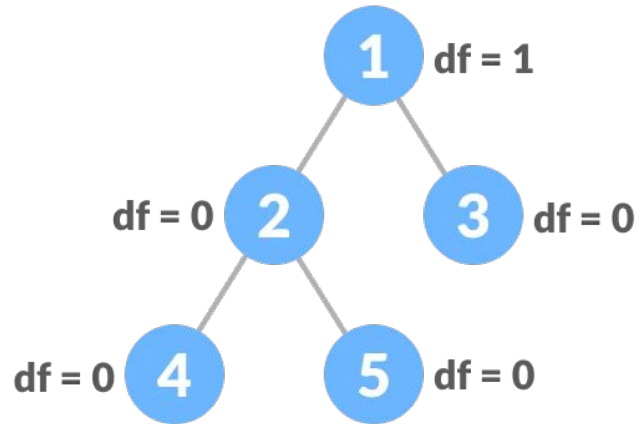
AVL Trees

WEEK -11

27-31/10/2025

Balance Binary Tree

A balanced binary tree, also referred to as a height-balanced binary tree, is defined as a binary tree in which the height of the left and right subtree of any node differ by not more than 1.

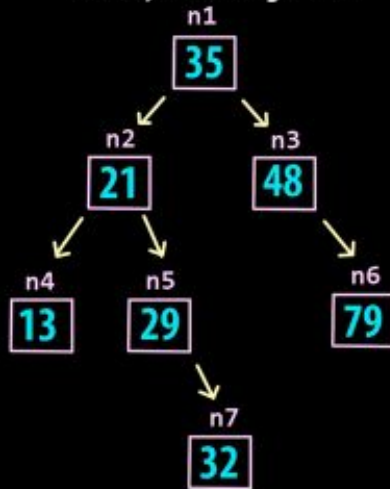


$df = |\text{height of left child} - \text{height of right child}|$

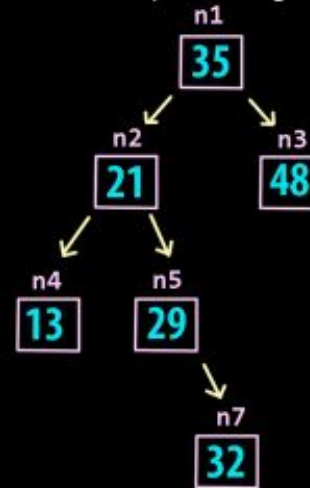
Balance Binary Tree

A Binary Tree is called a **BALANCED** binary tree, if the difference between the HEIGHT of left & right subtree for every node is not more than k (usually $k = 1$) Height of a tree is the number of edges on the longest path between root node & leaf node.

Binary Tree Diagram 1



Binary Tree Diagram 2



AVL TREE -Introduction

- ▶ AVL trees are self balancing Binary search trees.
- ▶ $\text{Height}(N) = \text{Max}(\text{Height}(\text{LST}), \text{Height}(\text{RST})) + 1$
- ▶ Balance Factor = $\text{Height}(\text{LST}) - \text{Height}(\text{RST})$
- ▶ Allowed values of BF are -1, 0, and +1.

Balance Factor	Meaning
0	Perfectly balanced
+1	Left heavy
-1	Right heavy
> +1 or < -1	 Unbalanced (rotation needed)

Type	Condition	Example Case
Left Rotation (LL)	Right heavy (BF = -2 and right->BF ≤ 0)	Right-Right
Right Rotation (RR)	Left heavy (BF = +2 and left->BF ≥ 0)	Left-Left
Left-Right Rotation (LR)	Left heavy + right child heavy	Left-Right
Right-Left Rotation (RL)	Right heavy + left child heavy	Right-Left

AVL TREE -Introduction

AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.

Avl operation :

1. Insert
2. Delete
3. Search

Ways of balancing AVL:

Right Right Rotation

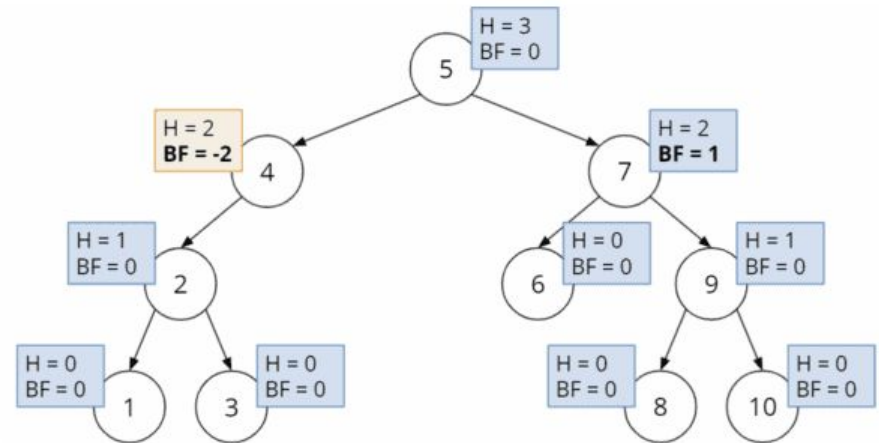
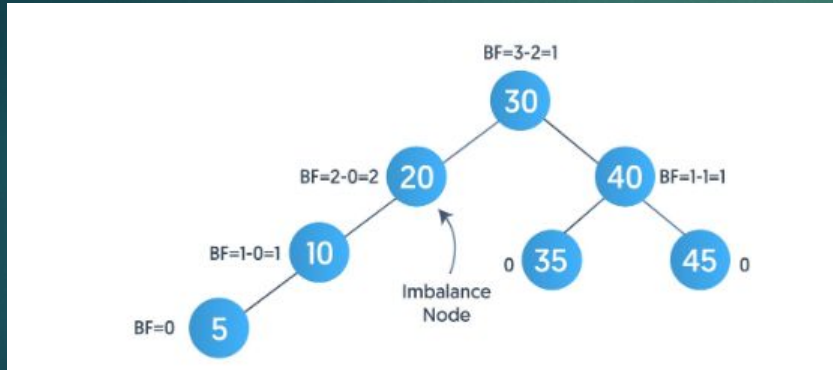
Left Left Rotation

Right Left Rotation

Left Right Rotation

Exercise

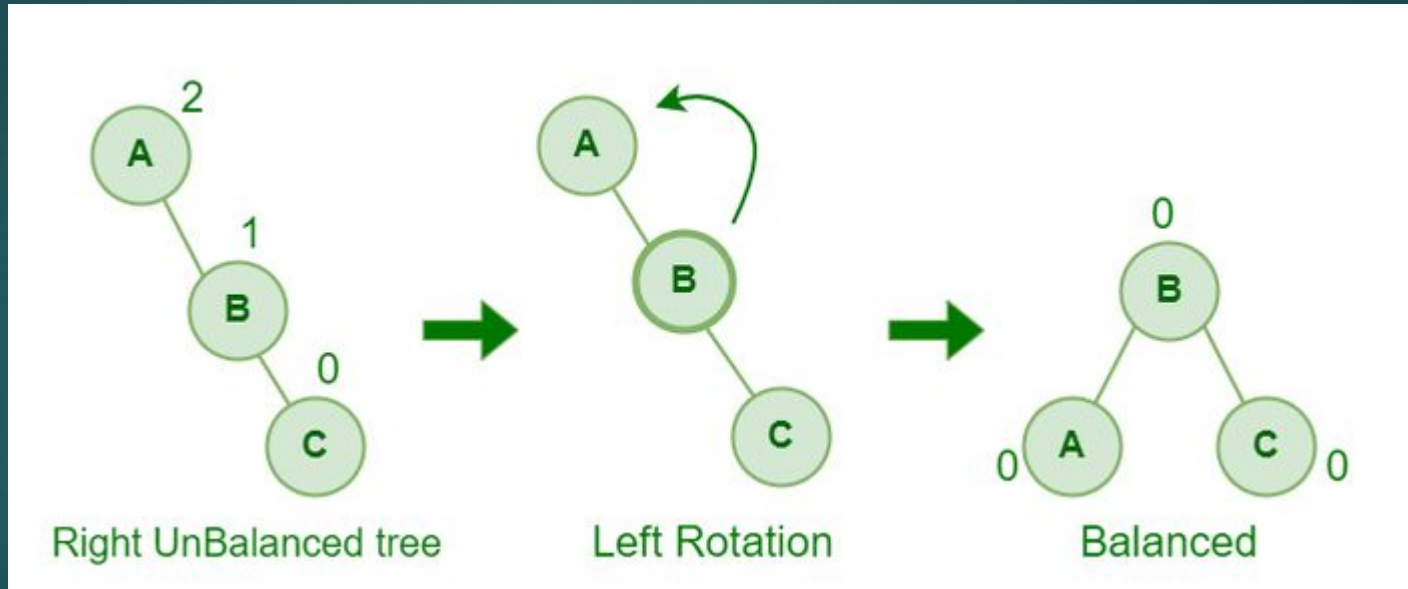
- Calculate heights and balance factors of the following trees.



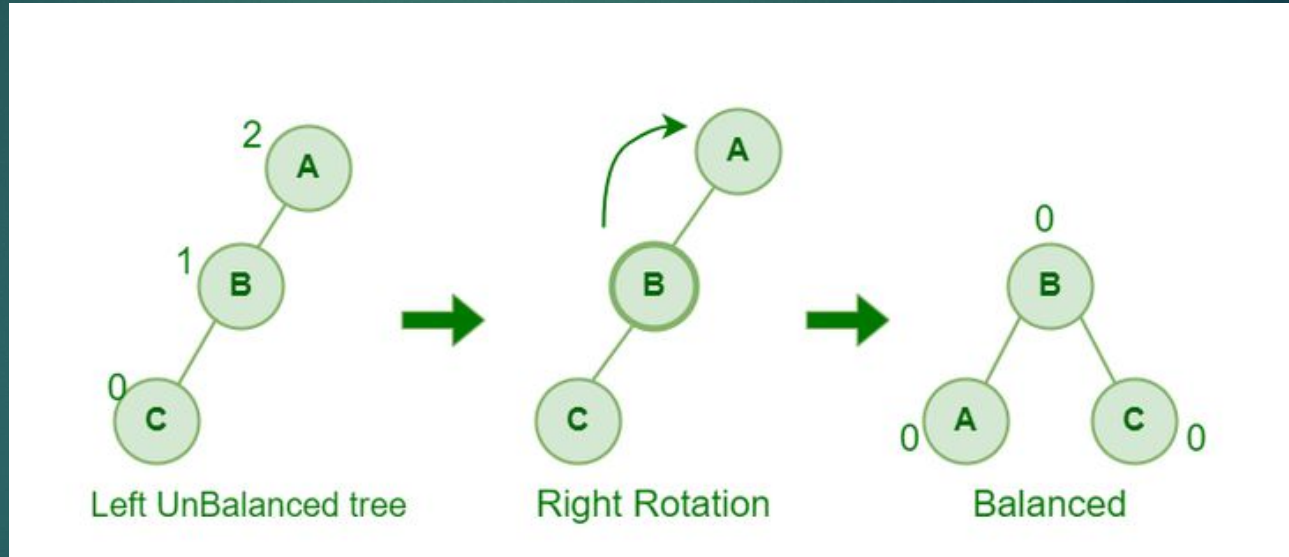
Binary search tree not satisfying the AVL invariant

Avl

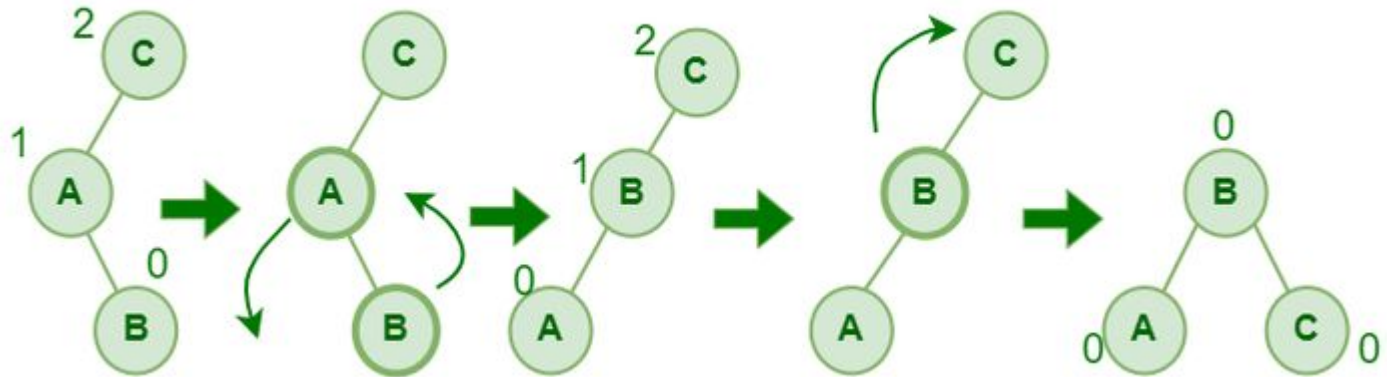
Left Rotation:



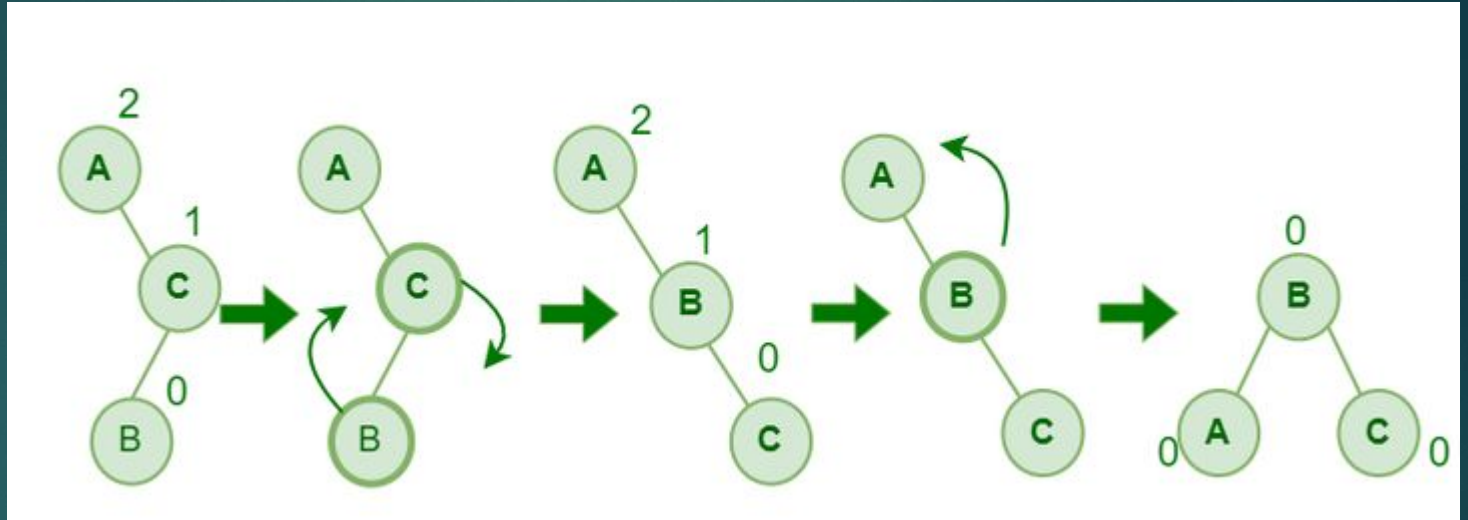
Right Rotation:



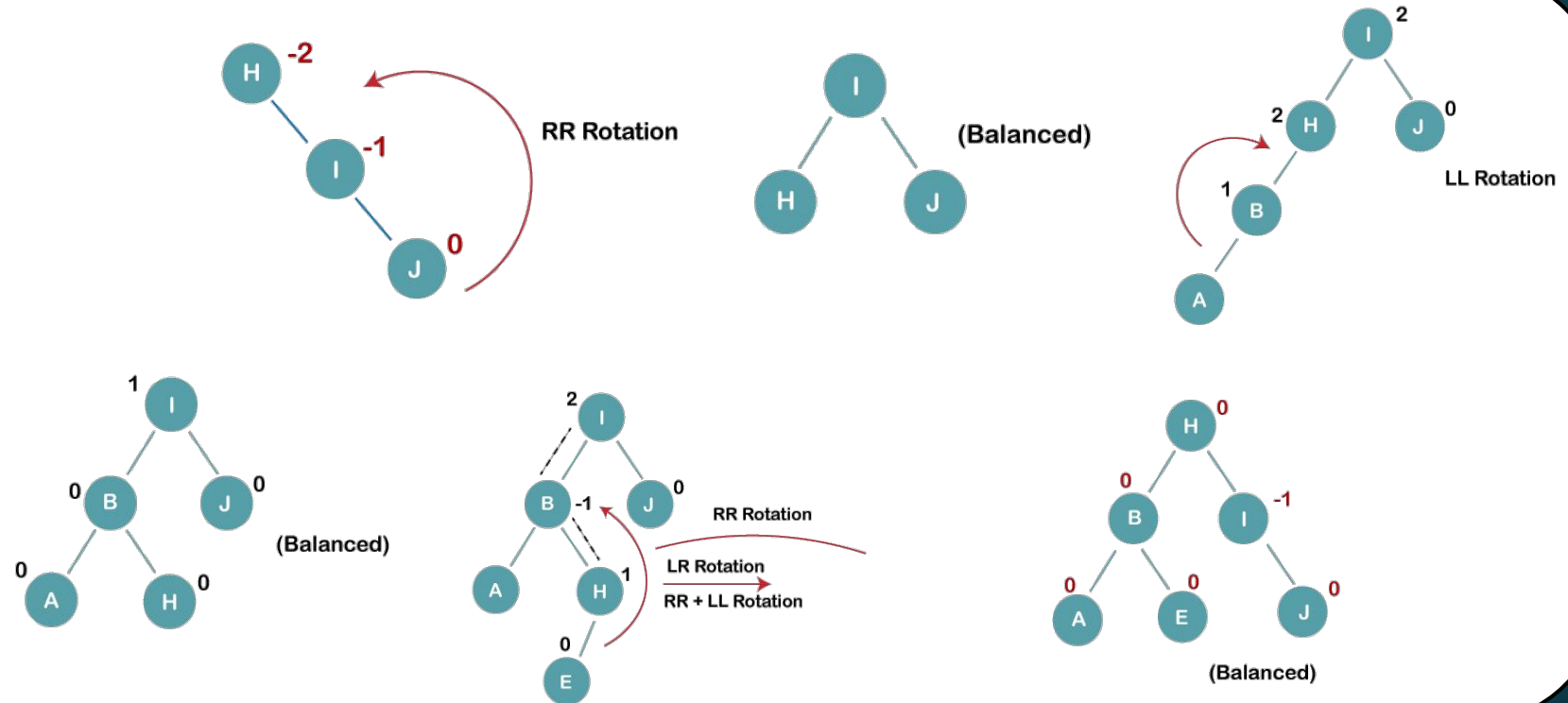
Left-Right Rotation



Right-Left Rotation



Construct an AVL tree having the following elements H, I, J, B, A, E, C, F, D, G, K, L



Exercise

Construct an AVL tree having the following elements

1. H, I, J, B, A, E, C, F, D, G, K, L

2. 14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20

Right rotate

```
Node *rightRotate(Node *y)
{
    Node *x = y->left;
    Node *T2 = x->right;

    // Perform rotation
    x->right = y;
    y->left = T2;

    // Update heights
    y->height = max(height(y->left),
                    height(y->right)) + 1;
    x->height = max(height(x->left),
                    height(x->right)) + 1;

    // Return new root
    return x;
}
```

Left Rotate

```
Node *leftRotate(Node *x)
{
    Node *y = x->right;
    Node *T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = max(height(x->left),
                    height(x->right)) + 1;
    y->height = max(height(y->left),
                    height(y->right)) + 1;

    // Return new root
    return y;
}
```


Insertion in AVL Algorithm

In the recursive BST insert, after insertion, we get pointers to all ancestors one by one in a bottom-up manner. So we don't need parent pointer to travel up. The recursive code itself travels up and visits all the ancestors of the newly inserted node.

- 1) Perform the normal BST insertion.
- 2) The current node must be one of the ancestors of the newly inserted node. Update the height of the current node.
- 3) Get the balance factor (left subtree height – right subtree height) of the current node.
- 4) If balance factor is greater than 1, then the current node is unbalanced and we are either in Left Left case or left Right case. To check whether it is left left case or not, compare the newly inserted key with the key in left subtree root.
- 5) If balance factor is less than -1, then the current node is unbalanced and we are either in Right Right case or Right-Left case. To check whether it is Right Right case or not, compare the newly inserted key with the key in right subtree root.

Implementation

```
Node* insert(Node* node, int key)
{
    /* 1. Perform the normal BST insertion */
    if (node == NULL)
        return(newNode(key));

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else // Equal keys are not allowed in BST
        return node;

    node->height = 1 + max(height(node->left),
                           height(node->right));
    int balance = getBalance(node);
}
```

```
// Left Left Case
if (balance > 1 && key < node->left->key)
    return rightRotate(node);

// Right Right Case
if (balance < -1 && key > node->right->key)
    return leftRotate(node);

// Left Right Case
if (balance > 1 && key > node->left->key)
{
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// Right Left Case
if (balance < -1 && key < node->right->key)
{
    node->right = rightRotate(node->right);
    return leftRotate(node);
}

/* return the (unchanged) node pointer */
return node;
}
```

Deletion in AVL Algorithm

- ▶ Let w be the node to be deleted
 - 1) Perform standard BST delete for w .
 - 2) Starting from w , travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the larger height child of z , and x be the larger height child of y . Note that the definitions of x and y are different from insertion.
 - 3) Re-balance the tree by performing appropriate rotations on the subtree rooted with z . There can be 4 possible cases that needs to be handled as x , y and z can be arranged in 4 ways. Following are the possible 4 arrangements:
 - a) y is left child of z and x is left child of y (Left Left Case)
 - b) y is left child of z and x is right child of y (Left Right Case)
 - c) y is right child of z and x is right child of y (Right Right Case)
 - d) y is right child of z and x is left child of y (Right Left Case)
- Like insertion, following are the operations to be performed in above mentioned 4 cases. Note that, unlike insertion, fixing the node z won't fix the complete AVL tree. After fixing z , we may have to fix ancestors of z as well

Deletion in AVL

For Deletion there are three cases:

1. If it is leaf node then just delete it
2. If node have one child then link the child with parent of node
3. If it node have two child then there are two case:
 - a. Find max node from left subtree
 - b. Or find min node from right subtree